

Livrable 2 - Nutrichain

CADET HUGO

DELAMARE Paul

GIMENEZ Yoann

I. Détails manquants lors du Livrable 1

A. Lien entre fonctionnalités clés, KPI et méthode de mesure

Chaque KPI est directement rattaché à une fonctionnalité critique du système, permettant de mesurer concrètement la valeur métier apportée par la plateforme et de piloter les améliorations continues.

Feature NutriChain	KPI associé	Objectif / Seuil	Méthode de mesure
Scan & traçabilité EPCIS (réception → expédition)	Couverture de traçabilité	> 98%	% de lots ayant un graphe complet (requête DB sur événements EPCIS)
Moteur de rappel produit ciblé	Temps médian de rappel	< 15 min	Timestamp entre décision de rappel et envoi notification
Pipeline IoT (capteurs → alertes)	Temps p95 ingest → alerte	< 30 sec	Mesure entre timestamp capteur et création alerte
Application mobile (scan opérateur)	Latence de scan	< 500 ms	Temps entre scan et réponse API (logs frontend + backend)
Détection anomalies chaîne du froid	Nombre d'excursions	< 15 / mois	Comptage mensuel des alertes générées
Système de notification (email/SMS/API)	Taux de réception des alertes	> 99%	% notifications envoyées vs reçues (logs + accusés)
Logs WORM & audit trail	Disponibilité des logs	> 99.9%	% logs disponibles sur 30 jours

B. Comparaison d'architecture (tableau clair)

Architecture	Avantages	Risques / Limites	Pertinence pour NutriChain
Monolithe 3-tiers	Simple à développer, rapide pour MVP	Couplage fort, faible scalabilité, difficile à maintenir	Peu adapté (complexité métier élevée)
Monolithe modulaire	Bonne séparation	Discipline nécessaire, point	Très adapté (MVP +

(hexagonal)	métier, testabilité, évolutivité progressive	de défaillance unique	évolution)
Microservices synchrones	Scalabilité, découplage fort, déploiement indépendant	Complexité élevée, coût infra, gestion réseau difficile	Possibilité d'être surdimensionné et incapacité de rendre un POC suffisamment complet

Le choix du monolithe modulaire avec architecture hexagonale représente un compromis optimal entre :

- simplicité de mise en œuvre (adapté au contexte académique),
- structuration métier robuste,
- capacité d'évolution vers des microservices à moyen terme.

C. Parties prenantes → tableau acteur / besoin / succès

La formalisation suivante permet de relier directement les besoins métier aux indicateurs de performance (KPI) et aux fonctionnalités implémentées dans Nutrichain.

Acteur	Besoin principal	Critère de réussite
Équipe Qualité	Identifier rapidement les lots à risque	Rappel lancé en < 15 min
Production	Enregistrer les transformations sans erreur	< 3% d'erreurs de scan
Logistique	Suivre les flux en temps réel	100% des événements EPCIS enregistrés
DSI	Intégration sécurisée et stable	> 99.9% disponibilité + MFA actif
Magasins	Vérifier rapidement les produits à retirer	Consultation lot < 2 sec
Consommateurs (B2C)	Vérifier si un produit est concerné par un rappel	Réponse via QR code en < 3 sec

II. Architecture retenue

Les exigences de ce projet impliquent une logique métier riche, fortement orientée événements, avec des règles fonctionnelles critiques (auditabilité, intégrité des données, historisation).

Dans ce contexte, le choix de l'architecture logicielle doit répondre à plusieurs objectifs :

- garantir une bonne structuration du métier,
- permettre une évolution progressive du système,
- rester réaliste et maîtrisable dans le cadre d'un projet académique réalisé par une équipe réduite.

L'architecture monolithique classique, bien que simple à mettre en œuvre, présente des limites importantes face à la diversité des domaines fonctionnels de NutriChain. Le regroupement de la traçabilité, de la gestion des alertes, de la chaîne du froid et du reporting dans un même noyau faiblement structuré entraîne un couplage fort, rendant l'évolution et la maintenance plus complexes.

À l'inverse, une architecture microservices, bien qu'adaptée à des systèmes industriels à grande échelle, introduit une complexité significative en termes d'infrastructure, de déploiement, de communication réseau et d'observabilité. Dans le cadre d'un projet académique, cette complexité représenterait un coût disproportionné par rapport à la valeur fonctionnelle réellement apportée, en particulier pour un MVP.

L'architecture monolithe modulaire permet de concilier les avantages des deux approches précédentes.

Elle repose sur une application unique, facilitant le déploiement et la mise en œuvre, tout en imposant une séparation stricte des domaines métiers.

Dans NutriChain, cette séparation se traduit naturellement par des modules correspondant aux grands domaines :

- gestion du catalogue et des référentiels,
- traçabilité des lots et événements EPCIS,
- supervision de la chaîne du froid,
- gestion des alertes et rappels produits,
- reporting et indicateurs.

Chaque module est conçu comme une unité cohérente, responsable de son propre périmètre fonctionnel, ce qui améliore la lisibilité du code et réduit les dépendances croisées.

L'intégration des principes de l'architecture hexagonale renforce encore la pertinence de ce choix.

En isolant le cœur métier des préoccupations techniques (API REST, base de données, services externes, simulation IoT), NutriChain bénéficie d'un modèle où la logique fonctionnelle reste indépendante des choix technologiques.

Cette approche est particulièrement adaptée au projet pour plusieurs raisons :

- la traçabilité et les règles métier EPCIS peuvent être testées indépendamment de l'infrastructure,
- les sources de données (capteurs simulés, entrées manuelles, API externes) peuvent évoluer sans remise en cause du métier,
- l'architecture facilite la simulation de composants (IoT, alertes), ce qui correspond aux contraintes pédagogiques du projet.

L'architecture hexagonale contribue également à une meilleure testabilité, en permettant de valider les cas d'usage métier sans dépendre d'une base de données ou d'un serveur HTTP.

Un autre avantage majeur de l'architecture monolithe modulaire avec architecture hexagonale est sa capacité à préparer une évolution vers une architecture plus distribuée, sans réécriture complète du système.

Les modules métiers, déjà bien délimités et faiblement couplés, peuvent être extraits ultérieurement sous forme de microservices si le contexte l'exige (montée en charge, exigences de disponibilité accrues, organisation par équipes).

Ainsi, l'architecture retenue s'inscrit dans une logique d'évolution progressive, en cohérence avec les pratiques industrielles.

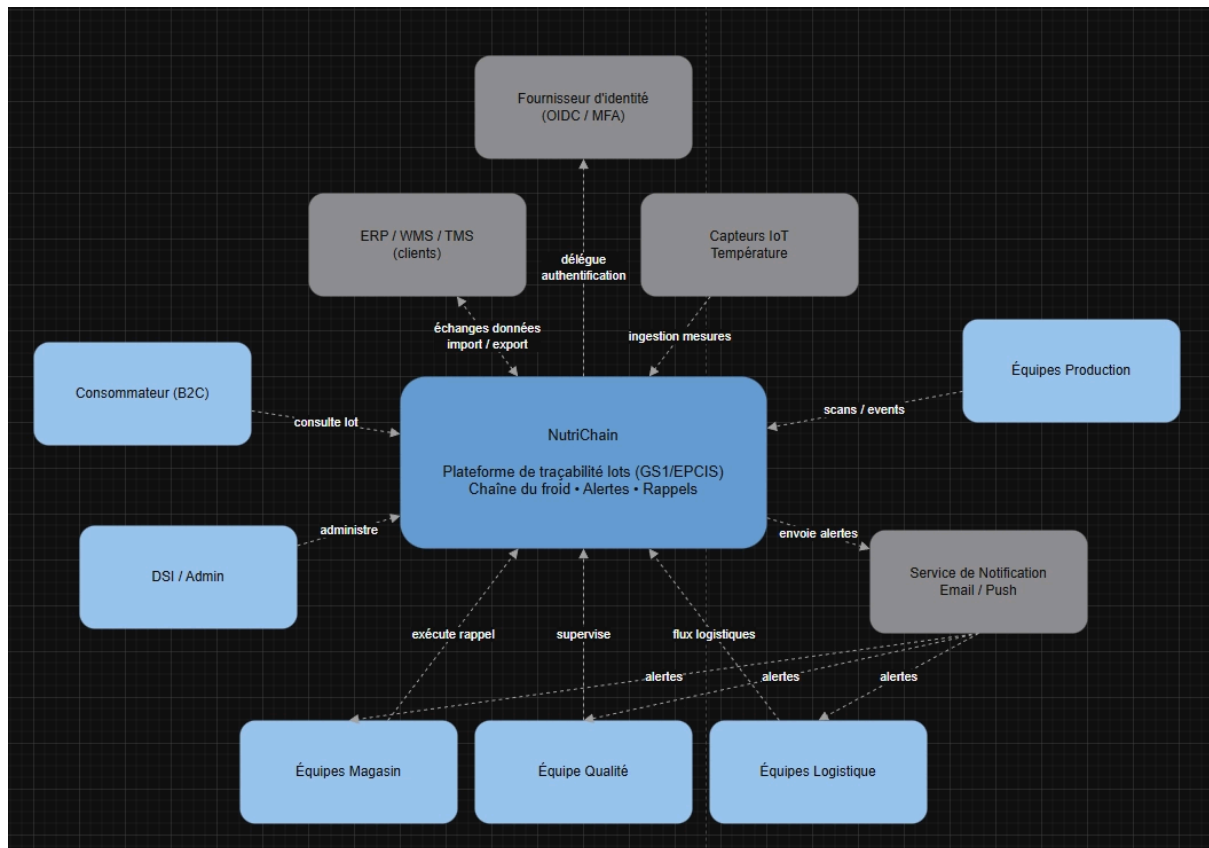
Au regard des exigences fonctionnelles de NutriChain, des contraintes opérationnelles décrites dans la synthèse détaillée et du cadre académique du projet, l'architecture monolithe modulaire s'appuyant sur les principes de l'architecture hexagonale apparaît comme le choix le plus pertinent.

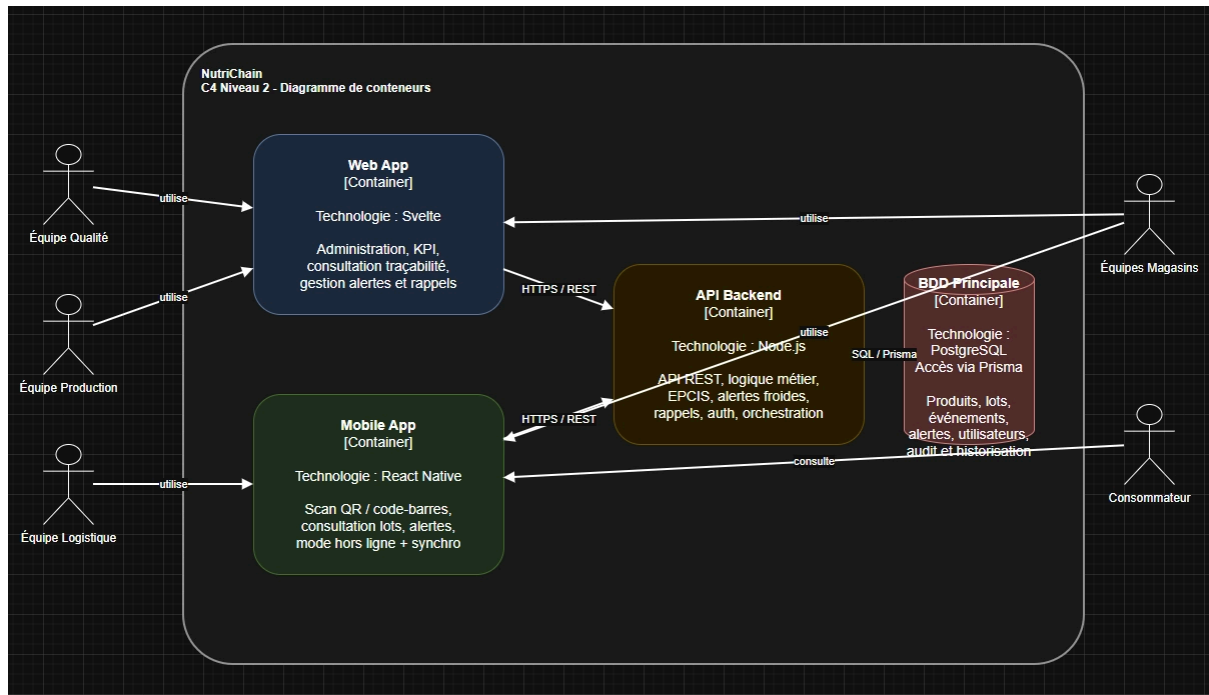
Elle permet :

- une structuration claire et robuste des domaines métier,
- une maîtrise de la complexité technique,

- une forte testabilité,
- et une évolutivité raisonnée vers des architectures plus distribuées.

Ce choix offre ainsi un équilibre optimal entre rigueur architecturale, faisabilité et crédibilité industrielle, en adéquation avec les objectifs du projet NutriChain.

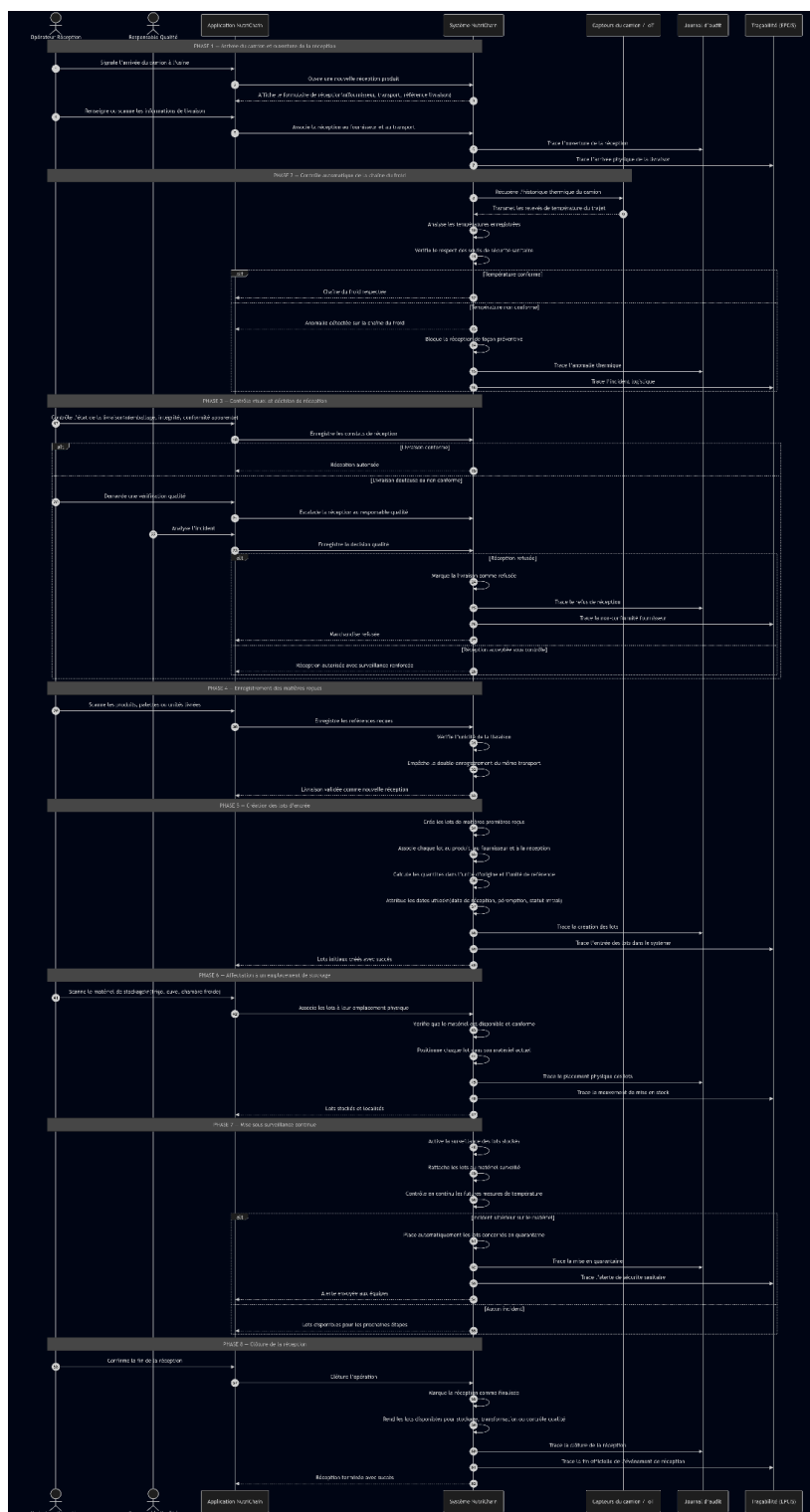




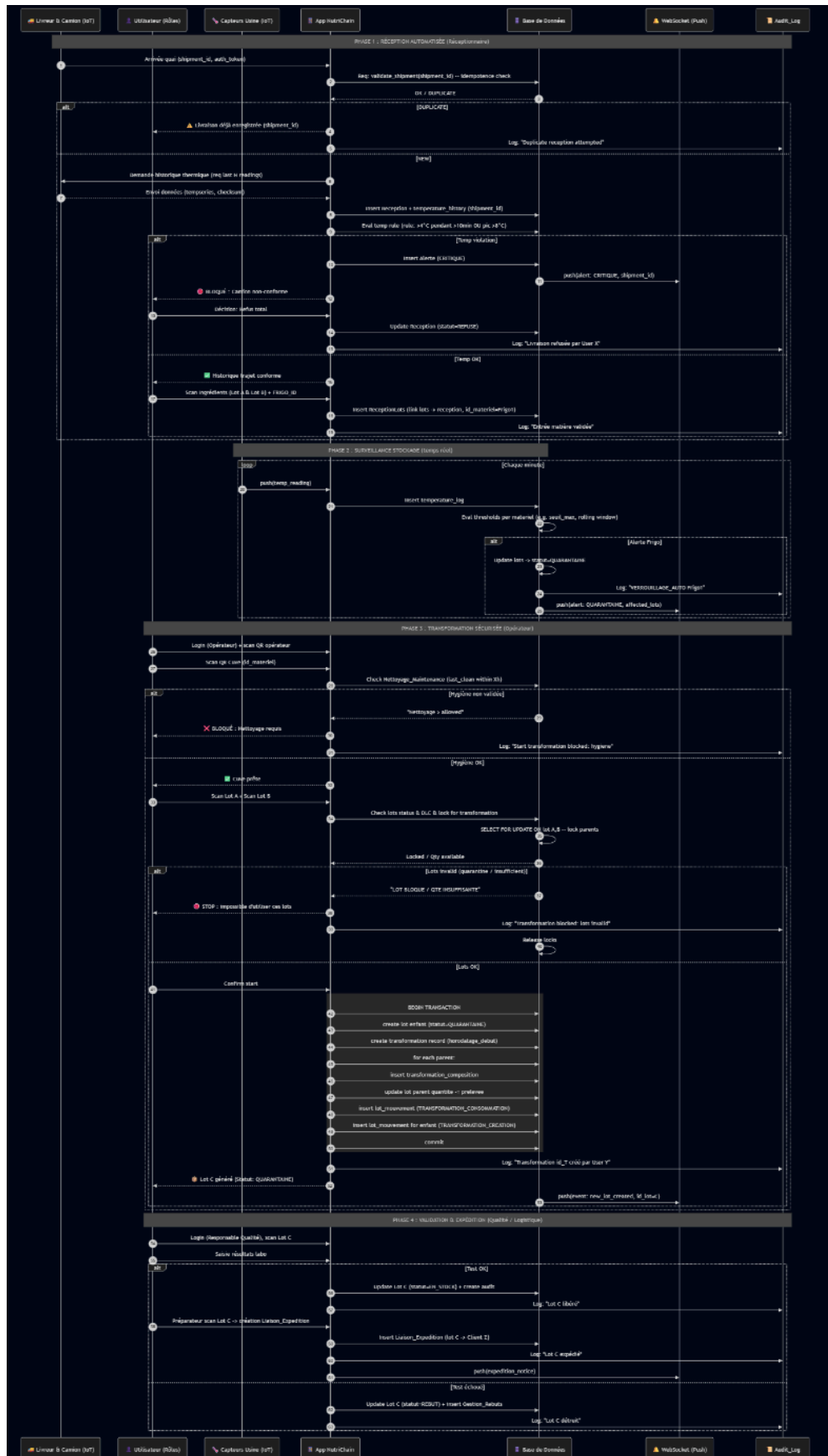
III. Modélisation de l'architecture et des bases de données

A. Processus de réception

[Lien du processus de réception produit](#)

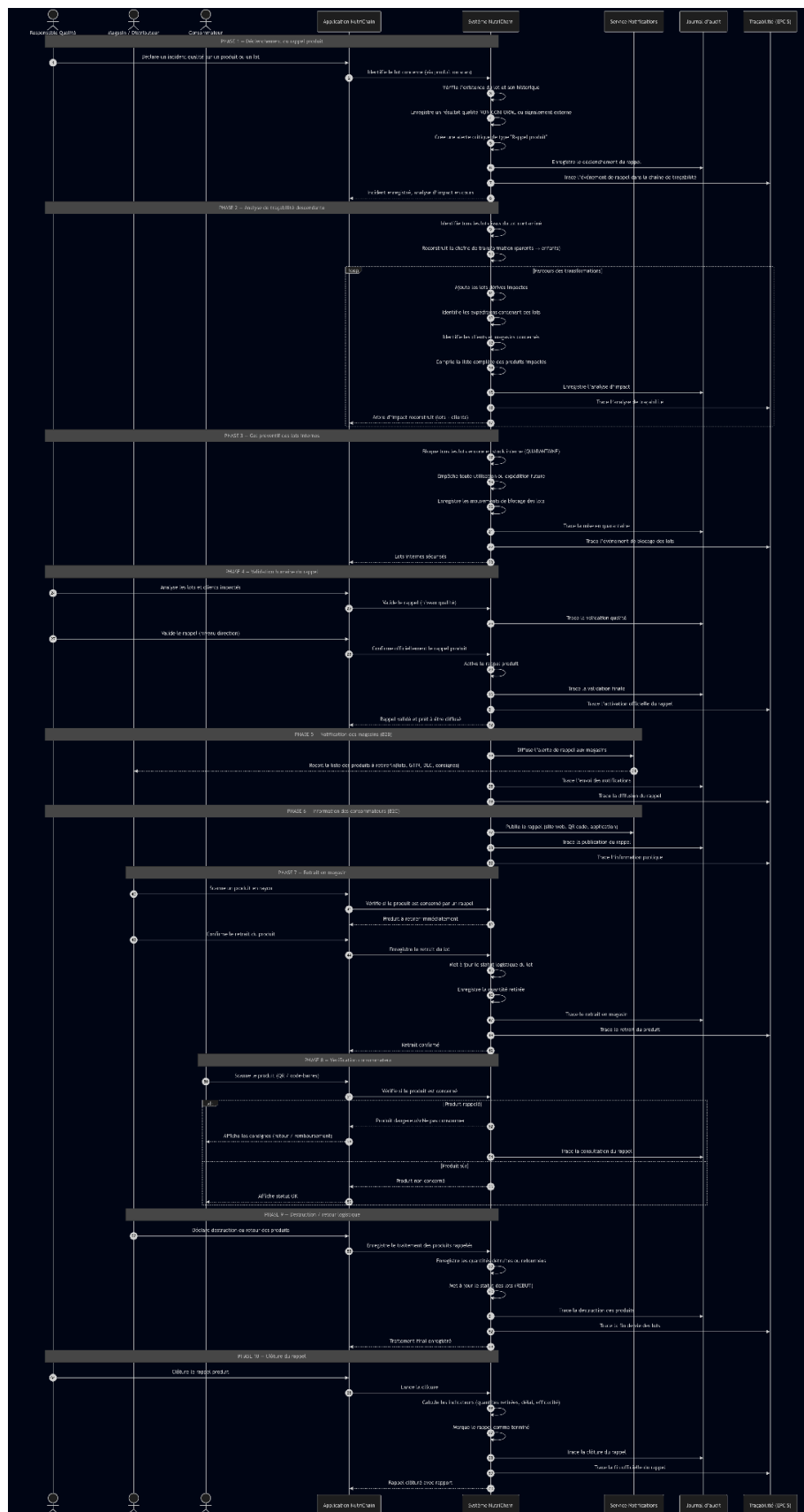


B. Processus de transformation



C. Processus de rappel produit

Lien du processus de rappel produit



D. Base de données

0. Tables utilitaires

- **Table `Unite`** (canonicalisation des unités) — *Liste des unités et facteurs de conversion*
 - `code` : Postgres `text` — Prisma `String` — Identifiant court de l'unité (ex: "L", "kg", "u").
 - `nom` : Postgres `text` — Prisma `String` — Nom lisible de l'unité.
 - `factor_to_base` : Postgres `numeric` — Prisma `Decimal` — Facteur de conversion vers l'unité de référence (pour normaliser les quantités).
 - **Table `EPCIS_Event`** (optionnel mais recommandé pour traçabilité événementielle) — *Events EPCIS pour reconstituer la chronologie métier*
 - `id` : Postgres `uuid` — Prisma `String (uuid)` — Identifiant unique de l'événement.
 - `event_time` : Postgres `timestampz` — Prisma `DateTime` — Horodatage de l'événement.
 - `event_type` : Postgres `text` / ENUM — Prisma `String` / Enum — Type d'événement (RECEPTION, TRANSFORMATION, ...).
 - `related_entity` : Postgres `text` — Prisma `String` — Table ou entité liée ("lot", "reception"...).
 - `related_id` : Postgres `text` — Prisma `String` — Identifiant de la ressource liée (uuid ou string).
 - `payload` : Postgres `jsonb` — Prisma `Json` — Données détaillées (métadonnées, mesures).
-

1. Table `Lieux` (Le plan de l'usine) — *Zones physiques de l'usine*

- `id` : Postgres `uuid` — Prisma `String (uuid)` — Identifiant unique du lieu.
 - `nom` : Postgres `text` — Prisma `String` — Nom lisible (ex : "Entrepôt Nord").
 - `type` : Postgres `text` / ENUM — Prisma `String` / Enum — Catégorie (Stockage, Transformation, Quai).
 - `description` : Postgres `text` — Prisma `String` — Informations complémentaires (accès, contraintes).
-

2. Table `Materiel` (L'inventaire technique) — *Machines, frigos, cuves*

- `id` : Postgres `uuid` — Prisma `String (uuid)` — Identifiant unique du matériel.
- `nom` : Postgres `text` — Prisma `String` — Libellé (ex : "Frigo n°3").

- `type` : Postgres `text` / ENUM — Prisma `String` / Enum — Catégorie (FROID, CHAUD, MIXEUR).
 - `id_lieu` : Postgres `uuid` — Prisma `String (uuid)` — FK → `Lieux.id`, localisation physique.
 - `statut` : Postgres `text` / ENUM — Prisma `String` / Enum — État opérationnel (PRET, EN_NETTOYAGE, ALERTE, EN_UTILISATION).
 - `temp_actuelle` : Postgres `numeric` — Prisma `Decimal` — Valeur cache de la température (source de vérité = `Temperature_Log`).
 - `temp_seuil_max` : Postgres `numeric` — Prisma `Decimal` — Seuil à partir duquel on déclenche alerte.
 - `qr_code_id` : Postgres `text` — Prisma `String` — Identifiant scannable pour attacher un opérateur.
 - `last_cleaned_at` : Postgres `timestamp` — Prisma `DateTime` — Timestamp du dernier nettoyage validé.
-

3. Table `Lot` (La "vie" du produit) — *Suivi d'un batch / lot physique*

- `id` : Postgres `uuid` — Prisma `String (uuid)` — Identifiant unique (ou GS1 si applicable).
 - `id_produit` : Postgres `uuid` — Prisma `String (uuid)` — FK → `Produit.id`.
 - `quantite_actuelle` : Postgres `numeric` — Prisma `Decimal` — Quantité restante dans l'unité `unite`.
 - `unite` : Postgres `text` — Prisma `String` — Référence `Unite.code`.
 - `quantite_base` : Postgres `numeric` — Prisma `Decimal` — Quantité convertie en unité de référence (pour checks).
 - `date_peremption` : Postgres `timestamp` — Prisma `DateTime` — DLC / DLUO.
 - `statut` : Postgres `text` / ENUM — Prisma `String` / Enum — (EN_STOCK, CONSOMME, EXPEDIE, QUARANTAINE, REBUT).
 - `id_materiel_actuel` : Postgres `uuid` — Prisma `String (uuid)` — FK → `Materiel.id` (position courante).
 - `date_creation` : Postgres `timestamp` — Prisma `DateTime` — Date de création du lot.
 - `created_by` : Postgres `uuid` — Prisma `String (uuid)` — FK → `User.id`.
 - `version` : Postgres `integer` — Prisma `Int` — Version pour optimistic locking (si utilisé).
-

4. Table `Transformation` (Le lien de parenté) — *Événement production (N parents → 1 enfant)*

- `id` : Postgres `uuid` — Prisma `String (uuid)` — Identifiant de l'opération.

- `id_lot_enfant` : Postgres `uuid` — Prisma `String (uuid)` — FK → `Lot.id` du lot créé.
 - `id_produit_fini` : Postgres `uuid` — Prisma `String (uuid)` — FK → `Produit.id` attendu.
 - `id_recette` : Postgres `uuid` (nullable) — Prisma `String (uuid)` — FK → `Recette_Composition.id` pour validation.
 - `id_user` : Postgres `uuid` — Prisma `String (uuid)` — FK → `User.id` (opérateur initiateur).
 - `id_materiel` : Postgres `uuid` — Prisma `String (uuid)` — FK → `Materiel.id` utilisé.
 - `statut` : Postgres `text` / ENUM — Prisma `String / Enum` — (EN_COURS, TERMINE, ANNULE, ERREUR).
 - `note_technique` : Postgres `jsonb` — Prisma `Json` — Paramètres machine et métadonnées.
 - `horodatage_debut` : Postgres `timestampz` — Prisma `DateTime` — Début de l'opération.
 - `horodatage_fin` : Postgres `timestampz` (nullable) — Prisma `DateTime` — Fin de l'opération.
 - `duration_seconds` : Postgres `integer` (nullable) — Prisma `Int` — Durée calculée (optionnel).
-

4bis. Table `Transformation_Composition` (Pivot) — *Détail des prélèvements pour une transformation*

- `id_transformation` : Postgres `uuid` — Prisma `String (uuid)` — FK → `Transformation.id`.
 - `id_lot_parent` : Postgres `uuid` — Prisma `String (uuid)` — FK → `Lot.id` parent.
 - `quantite_prelevee` : Postgres `numeric` — Prisma `Decimal` — Quantité prélevée (dans `unite`).
 - `unite` : Postgres `text` — Prisma `String` — `Unite.code`.
 - `lot_parent_epuise` : Postgres `boolean` — Prisma `Boolean` — True si le parent est vidé après prélèvement.
 - `note` : Postgres `text` — Prisma `String` — Commentaire / conversion appliquée.
-

5. Table `User & Role` (Sécurité) — *Comptes utilisateurs et rôle*

- `id` : Postgres `uuid` — Prisma `String (uuid)` — Identifiant user.
- `nom_prenom` : Postgres `text` — Prisma `String` — Nom complet.
- `email` : Postgres `text` — Prisma `String` — Adresse email (unique côté application).

- `password_hash` : Postgres `text` — Prisma `String` — Hash du mot de passe.
- `id_role` : Postgres `uuid` — Prisma `String (uuid)` — FK → `Role.id`.
- `mfa_enabled` : Postgres `boolean` — Prisma `Boolean` — MFA activé ou non.
- `last_login` : Postgres `timestamptz` — Prisma `DateTime` — Dernière connexion.

(Table `Role` : `id uuid`, `name text` — ex: `OPERATOR`, `RECEPTION`, `QUALITE`, `ADMIN`.)

6. Table `Produit` (Catalogue) — *Référentiel produit et conservation*

- `id` : Postgres `uuid` — Prisma `String (uuid)` — Identifiant produit.
 - `nom` : Postgres `text` — Prisma `String` — Libellé produit (Yaourt nature).
 - `code_gtin` : Postgres `text` — Prisma `String` — GTIN / code-barres GS1.
 - `categorie` : Postgres `text` — Prisma `String` — Catégorie (Produit fini, Matière première...).
 - `duree_conservation_default` : Postgres `integer` — Prisma `Int` — Durée en jours pour DLC par défaut.
 - `seuil_alerte_stock` : Postgres `numeric` — Prisma `Decimal` — Seuil commande/réappro.
 - `unite_reference` : Postgres `text` — Prisma `String` — `Unite.code` référence.
-

7. Table `Expedition` — *Bon de livraison / shipment*

- `id` : Postgres `uuid` — Prisma `String (uuid)` — Identifiant expédition.
 - `id_client` : Postgres `uuid` — Prisma `String (uuid)` — FK → `Client.id`.
 - `shipment_id` : Postgres `text (UNIQUE)` — Prisma `String` — Identifiant transporteur (SSCC) pour idempotence.
 - `date_envoi` : Postgres `timestamptz` — Prisma `DateTime` — Date et heure d'envoi.
 - `transporteur` : Postgres `text` — Prisma `String` — Nom du transporteur.
 - `statut_livraison` : Postgres `text / ENUM` — Prisma `String` — (PREPARATION, EN_ROUTE, LIVRE, RETOURNE).
 - `created_by` : Postgres `uuid` — Prisma `String (uuid)` — FK → `User.id`.
-

8. Table `Client` — *Destinataires / points de vente*

- `id` : Postgres `uuid` — Prisma `String (uuid)` — Identifiant client.
- `nom_enseigne` : Postgres `text` — Prisma `String` — Nom commercial.

- `contact_urgence` : Postgres `text` — Prisma `String` — Téléphone / email responsable qualité.
 - `adresse_livraison` : Postgres `text` — Prisma `String` — Adresse complète.
 - `notes` : Postgres `text` — Prisma `String` — Notes opérationnelles (SLA, contraintes).
-

9. Table `Alerte` (La réactivité 30s) — *Alertes critiques & notifications*

- `id` : Postgres `uuid` — Prisma `String (uuid)` — Identifiant alerte.
- `type` : Postgres `text` / ENUM — Prisma `String` — TEMPÉRATURE / DLC_PROCHE / ...
- `niveau_gravite` : Postgres `text` / ENUM — Prisma `String` — INFO / WARNING / CRITIQUE.
- `message` : Postgres `text` — Prisma `String` — Texte descriptif de l'alerte.
- `id_materiel` : Postgres `uuid` (nullable) — Prisma `String (uuid)` — FK → `Materiel.id` si applicable.
- `related_entity` : Postgres `text` — Prisma `String` — Entité liée (ex: "lot").
- `related_id` : Postgres `text` — Prisma `String` — Identifiant lié.
- `statut` : Postgres `text` / ENUM — Prisma `String` — ACTIVE / ACQUITTEE / RESOLUE.
- `created_at` : Postgres `timestampz` — Prisma `DateTime` — Création de l'alerte.
- `resolved_by` : Postgres `uuid` (nullable) — Prisma `String (uuid)` — FK → `User.id`.
- `resolved_at` : Postgres `timestampz` (nullable) — Prisma `DateTime` — Résolution horodatée.

Comportement : DB/worker évalue les règles (ex: règle température) et crée Alerte. Push WebSocket sur création.

10. Table `Audit_Log` — *Journal immuable d'actions*

- `id` : Postgres `bigserial` — Prisma `Int` — Identifiant séquentiel.
- `id_user` : Postgres `uuid` (nullable) — Prisma `String (uuid)` — Qui a effectué l'action.
- `action` : Postgres `text` — Prisma `String` — Libellé action (ex: "Modification statut").
- `entity` : Postgres `text` — Prisma `String` — Table impactée.
- `entity_id` : Postgres `text` — Prisma `String` — Identifiant de la ligne affectée.
- `ancienne_valeur` : Postgres `jsonb` — Prisma `Json` — Valeur avant modification.
- `nouvelle_valeur` : Postgres `jsonb` — Prisma `Json` — Valeur après modification.

- `prev_hash` : Postgres `text` — Prisma `String` — Hash du log précédent (chainage).
 - `signature_hash` : Postgres `text` — Prisma `String` — Hash courant (intégrité).
 - `horodatage` : Postgres `timestampz` — Prisma `DateTime` — Horodatage.
-

11. Table `Liaison_Expedition` — *Contenu du camion / mapping lot ↔ expédition*

- `id` : Postgres `uuid` — Prisma `String (uuid)` — Identifiant.
 - `id_expedition` : Postgres `uuid` — Prisma `String (uuid)` — FK → `Expedition.id`.
 - `id_lot` : Postgres `uuid` — Prisma `String (uuid)` — FK → `Lot.id`.
 - `quantite_expediee` : Postgres `numeric` — Prisma `Decimal` — Quantité embarquée.
 - `unite` : Postgres `text` — Prisma `String` — `Unite.code`.
 - `pallet_id` : Postgres `text` (nullable) — Prisma `String` — Identifiant palette / agrégation.
-

12. Table `Fournisseur` — *Origine amont / producteur*

- `id` : Postgres `uuid` — Prisma `String (uuid)` — Identifiant fournisseur.
 - `nom_ferme` : Postgres `text` — Prisma `String` — Nom ferme/producteur.
 - `type_produit` : Postgres `text` — Prisma `String` — (Bio, AOP, Conventionnel).
 - `contact_qualite` : Postgres `text` — Prisma `String` — Contact qualité (tel/email).
 - `adresse_siege` : Postgres `text` — Prisma `String` — Adresse siège social.
-

13. Table `Reception` — *Entrée en usine / réception camion*

- `id` : Postgres `uuid` — Prisma `String (uuid)` — Identifiant réception.
- `id_fournisseur` : Postgres `uuid` — Prisma `String (uuid)` — FK → `Fournisseur.id`.
- `shipment_id` : Postgres `text` (UNIQUE NOT NULL) — Prisma `String` — Identifiant transporteur pour idempotence.
- `date_reception` : Postgres `timestampz` — Prisma `DateTime` — Date et heure d'arrivée.
- `temperature_camion_summary` : Postgres `jsonb` — Prisma `Json` — Résumé min/max/avg.
- `temperature_camion_raw` : Postgres `jsonb` (nullable) — Prisma `Json` — Séries brutes (optionnel).

- `statut_controle` : Postgres `text` / ENUM — Prisma `String` — (CONFORME, REFUSE, EN_ATTENTE_ANALYSE).
- `received_by` : Postgres `uuid` — Prisma `String (uuid)` — FK → `User.id`.

Processus : idempotence via `shipment_id`; si non-conforme → Alerte CRITIQUE
+ `Audit_Log`; création des Lot initiaux à l'arrivée.

14. Table `Nettoyage_Maintenance` — *Hygiène & interventions*

- `id` : Postgres `uuid` — Prisma `String (uuid)` — Identifiant intervention.
 - `id_materiel` : Postgres `uuid` — Prisma `String (uuid)` — FK → `Materiel.id`.
 - `id_user` : Postgres `uuid` — Prisma `String (uuid)` — FK → `User.id`.
 - `type_action` : Postgres `text` — Prisma `String` — Nettoyage complet / Désinfection / Maintenance.
 - `produit_utilise` : Postgres `text` — Prisma `String` — Produit chimique utilisé.
 - `date_debut` : Postgres `timestampz` — Prisma `DateTime` — Début intervention.
 - `date_fin` : Postgres `timestampz` — Prisma `DateTime` — Fin intervention.
 - `valid_until` : Postgres `timestampz` — Prisma `DateTime` — Validité du nettoyage (ex: 24h).
-

15. Table `Performance Stats` — *Métriques & KPIs*

- `id` : Postgres `bigserial` — Prisma `Int` — Identifiant métrique.
- `type_metrrique` : Postgres `text` — Prisma `String` — TEMPS_SCAN, LATENCEALERTE, etc.
- `valeur` : Postgres `numeric` — Prisma `Decimal` — Valeur mesurée.
- `id_reference` : Postgres `uuid` (nullable) — Prisma `String (uuid)` — Référence optionnelle (lot/alerte).
- `horodatage` : Postgres `timestampz` — Prisma `DateTime` — Horodatage métrique.

Recomm. stocker ces séries en TimescaleDB si possible.

16. Table `Gestion_Rebuts` — *Pertes / déchets*

- `id` : Postgres `uuid` — Prisma `String (uuid)` — Identifiant.
- `id_lot` : Postgres `uuid` — Prisma `String (uuid)` — FK → `Lot.id`.
- `quantite_jetee` : Postgres `numeric` — Prisma `Decimal` — Quantité mise au rebut.

- `unite` : Postgres `text` — Prisma `String` — `Unite.code`.
 - `motif` : Postgres `text` — Prisma `String` — Raison du rebut.
 - `date_mise_au_rebut` : Postgres `timestampz` — Prisma `DateTime` — Date du rebut.
 - `created_by` : Postgres `uuid` — Prisma `String (uuid)` — Opérateur ayant enregistré.
-

17. Table `Controle_Qualite` — *Résultats laboratoire*

- `id` : Postgres `uuid` — Prisma `String (uuid)` — Identifiant du test.
- `id_lot` : Postgres `uuid` — Prisma `String (uuid)` — FK → `Lot.id`.
- `type_test` : Postgres `text` — Prisma `String` — Type de test (Listeria, pH...).
- `resultat` : Postgres `text` / ENUM — Prisma `String` / Enum — CONFORME / NON_CONFORME.
- `id_user_labo` : Postgres `uuid` — Prisma `String (uuid)` — Technicien laboratoire.
- `certificat_pdf` : Postgres `text` — Prisma `String` — URL/chemin du document.
- `date_test` : Postgres `timestampz` — Prisma `DateTime`.
- `notes` : Postgres `text` — Prisma `String`.

Règle métier : lot enfant passe QUARANTAINE → EN_STOCK uniquement si `resultat = CONFORME`.

18. Table `Temperature_Log` (Historique) — *Séries temporelles IoT*

- `id` : Postgres `bigserial` — Prisma `Int` — Identifiant séquentiel.
- `id_materiel` : Postgres `uuid` — Prisma `String (uuid)` — FK → `Materiel.id`.
- `valeur` : Postgres `numeric` — Prisma `Decimal` — Valeur mesurée (°C).
- `unite` : Postgres `text` — Prisma `String` — Unité (ex: "C").
- `horodatage` : Postgres `timestampz` — Prisma `DateTime` — Timestamp lecture.

Recomm. hypertable / partitioning + retention policy.

19. Table `Recette_Composition` (Garde-fou) — *Recettes théoriques et tolérances*

- `id` : Postgres `uuid` — Prisma `String (uuid)` — Identifiant recette.
- `id_produit_fini` : Postgres `uuid` — Prisma `String (uuid)` — FK → `Produit.id`.

- `id_type_ingredient` : Postgres `uuid` / `text` — Prisma `String` — Catégorie ingrédient (Lait, Ferment...).
- `quantite_theorique` : Postgres `numeric` — Prisma `Decimal` — Quantité attendue.
- `unite` : Postgres `text` — Prisma `String` — `Unite.code`.
- `tolerance_percent` : Postgres `numeric` — Prisma `Decimal` — Tolérance acceptée en %.

20. Table `Lot_Mouvement` (nouveau, essentiel) — *Historique append-only des mouvements*

- `id` : Postgres `bigserial` — Prisma `Int` — Identifiant séquentiel.
- `id_lot` : Postgres `uuid` — Prisma `String (uuid)` — FK → `Lot.id`.
- `type_action` : Postgres `text` — Prisma `String` — RECEPTION / TRANSFORMATION_CONSOMMATION / ...
- `quantite` : Postgres `numeric` — Prisma `Decimal` — Quantité impactée.
- `unite` : Postgres `text` — Prisma `String` — `Unite.code`.
- `id_transformation` : Postgres `uuid` (nullable) — Prisma `String (uuid)` — FK → `Transformation.id`.
- `id_expedition` : Postgres `uuid` (nullable) — Prisma `String (uuid)` — FK → `Expedition.id`.
- `id_user` : Postgres `uuid` (nullable) — Prisma `String (uuid)` — Opérateur initiateur.
- `created_at` : Postgres `timestampz` — Prisma `DateTime` — Horodatage écriture mouvement.
- `metadata` : Postgres `jsonb` — Prisma `Json` — Données additionnelles (raison, références).

Rôle : append-only — base de la reconstruction de l'historique et des audits.

NutriChain a été conçu pour digitaliser l'intégralité du cycle de vie des produits, **de la ferme au rayon**, en garantissant traçabilité, intégrité et actionnabilité en cas d'incident. Le point d'entrée est la chaîne logistique amont : chaque fournisseur est répertorié et chaque livraison donne lieu à une entrée structurée (**Reception**) identifiée de façon idempotente par un `shipment_id` (SSCC/BL). Cette contrainte d'unicité empêche le double-enregistrement des réceptions et constitue la première garantie contre les erreurs humaines et les replays IoT. Lors d'une réception, les lectures de température du camion sont stockées (résumé + lectures brutes si nécessaire) et évaluées selon une règle paramétrable ; si la règle est violée (par exemple : > 4 °C pendant ≥ 10 minutes consécutives ou un pic > 8 °C), le système crée immédiatement une **Alerte** critique, marque la réception comme **REFUSE** et génère les événements de traçabilité

(**EPCIS_Event**) et d'audit (**Audit_Log**). Les lots initiaux créés à la réception sont associés à la source (fournisseur) et contiennent quantité, unité et quantité convertie en unité de référence pour faciliter la vérification des bilans matières.

La table **Lot** est la vérité opérationnelle des quantités : elle contient **quantite_actuelle**, son unité d'origine et une **quantite_base** canonique. Toute modification de quantité ou de statut s'accompagne obligatoirement d'un enregistrement append-only dans **Lot_Mouvement** pour permettre la reconstruction complète de l'historique matériel. Les règles métier appliquées sont strictes : on ne modifie jamais un lot sans écrire un mouvement, on ne supprime pas ces mouvements et on applique des checks pour empêcher des quantités négatives. Un lot passe en **CONSOMME** quand sa **quantite_actuelle** atteint zéro ; un lot enfant créé lors d'une production est inséré initialement avec le statut **QUARANTAINE** et ne peut être libéré pour la vente qu'après validation explicite du laboratoire (**Controle_Qualite** = CONFORME).

La transformation est un événement métier atomique qui relie N lots parents à un lot enfant. Le processus doit être encapsulé dans une transaction courte et verrouiller les parents (**SELECT ... FOR UPDATE**) pour éviter le sur-prélèvement concurrentiel. Le flux idéal : verrouillage des lots parents, conversion des quantités prélevées en unités de référence via la table **Unite**, vérification de suffisance, insertion du lot enfant (**statut=QUARANTAINE**), insertion du **Transformation** et des lignes **Transformation_Composition**, mise à jour des quantités parents et écriture des **Lot_Mouvement** correspondants, puis commit. Si la validation de recette (**Recette_Composition**) échoue (hors tolérance), la transformation doit produire une alerte qualité et laisser le lot enfant en quarantaine — elle ne doit pas être automatiquement libérée. Si une erreur se produit pendant la transaction, le rollback annule tout ; si un incident survient après le commit, une procédure de compensation claire (script idempotent qui recréé parents et injecte **Lot_Mouvement(type=AJUSTEMENT)**) doit être disponible.

La gestion des unités est non négociable : toutes les conversions s'effectuent à l'écriture en utilisant **Unite.factor_to_base** et la **quantite_base** est persistée. Les contrôles empêchent l'écriture d'un prélèvement supérieur à la disponibilité (après conversion). Cette approche évite les erreurs de calculs à la volée et facilite la vérification des bilans en batch. Par ailleurs, les recettes doivent inclure une **tolerance_percent** : lors d'une transformation, on compare la composition réelle avec la théorique et l'on déclenche une alerte ou une mise en erreur si la déviation dépasse la tolérance documentée.

L'alerte et la surveillance temps réel reposent sur une ingestion sécurisée des données IoT : chaque capteur envoie des lectures signées/authentifiées vers le service d'ingestion, qui écrit **Temperature_Log** (hypertable ou partitions journalières recommandées) et émet les évaluations de règles sur des fenêtres glissantes. Lorsqu'une condition d'alerte est rencontrée, le worker crée une **Alerte**, met à jour le statut des lots affectés (**QUARANTAINE**), écrit les **Lot_Mouvement** nécessaires et pousse une notification via WebSocket / push mobile aux opérateurs concernés. La latence cible pour la chaîne ingestion→alerte doit être inférieure à 30 s pour respecter les exigences métier.

La traçabilité et l'intégrité des logs sont assurées par deux mécanismes complémentaires : **EPCIS_Event** pour les événements métier interopérables et **Audit_Log** append-only avec chaînage cryptographique (**prev_hash** + **signature_hash**). Des triggers **AFTER INSERT/UPDATE/DELETE** sur les tables critiques (Lot, Reception, Transformation, Expedition, Alerte) doivent générer automatiquement ces entrées d'audit. Les permissions DB doivent interdire toute suppression ou modification directe d'un audit ; les clés de signature sont gérées via un secret manager externe pour garantir l'intégrité hors de la base.

L'expédition est structurée autour d'un **shipment_id** unique et d'une table de jointure **Liaison_Expedition** reliant lots et expéditions (avec possibilité d'agrégation par **pallet_id**). Pour permettre un rappel rapide, la base maintient une table dérivée **lot_current_location** (mise à jour à chaque expédition) indexée sur **id_lot** : en cas d'alerte produit, on interroge directement cette table pour retrouver les clients impactés en temps constant/rapide. Le scénario de rappel documentaire : identification du lot suspect, récupération des clients via **lot_current_location**, création d'un **EPCIS_Event(RECALL)** et d'une **Alerte** CRITIQUE, blocage des préparations en cours et envoi de notifications aux clients et autorités selon SLA.

Sur la partie performance et maintien, les tables de séries temporelles (**Temperature_Log**, **Performance Stats**) doivent être optimisées via TimescaleDB ou partitioning, avec des politiques de rétention (ex. 2 ans) et routage vers un data lake (Parquet) pour l'archivage à long terme. Les tables volumineuses (**Lot_Mouvement**) doivent être partitionnées (mois/année) si le débit l'exige. Indices essentiels : **IDX_Lot(id_produit, statut)**, **IDX_Liaison_Expedition(id_lot)**, **IDX_Temperature_Log(id_materiel, horodatage DESC)** et **IDX_Lot_Mouvement(id_lot, created_at DESC)**. Ces choix permettent aux requêtes critiques (rappel, recherche lot→client, agrégations temporelles) d'être réalisées en quelques millisecondes.

La robustesse opérationnelle passe par des workers découplés : ingestion IoT, évaluation de règles, réconciliation mass balance, purge/archivage, et compensation des transformations. Ces workers doivent consommer depuis une file (Kafka/Rabbit) afin d'éviter de bloquer les transactions utilisateur et permettre un redémarrage sur événements non traités. Les triggers DB n'ont pas à contenir de logique métier lourde ; ils servent à garantir l'écriture d'événements d'audit et à alimenter la file.

Côté sécurité et gouvernance, applique RBAC fin : rôles **Reception**, **Operateur**, **Qualite**, **Logistique**, **Admin**. Restreint l'accès aux opérations sensibles via RLS et MFA pour **Qualite** et **Admin**. Les backups sont quotidiens avec WAL shipping pour un RPO faible ; les exports vers data lake servent d'archive immuable. Conformément aux exigences réglementaires, conserver les logs d'audit pendant la durée légale, chiffrés et horodatés de manière infalsifiable.

Enfin, prépare la montée en charge et les tests : jeux de tests unitaires DB (transactions de transformation, conversions d'unités), tests d'intégration bout-à-bout (réception → transformation → QC → expédition → rappel), tests de charge sur ingestion IoT et mesure de la latence d'alerte (p95 < 30 s). Documente les procédures opératoires pour les cas

d'erreur (transformation interrompue, réception refusée, dérive de mass balance) et fournis des scripts idempotents de compensation. Avec ces règles et mécanismes en place, NutriChain devient un système traçable, défendable en audit sanitaire, et capable d'exécuter un rappel produit rapide et fiable tout en gardant une base de données cohérente et fidèle à la réalité terrain.

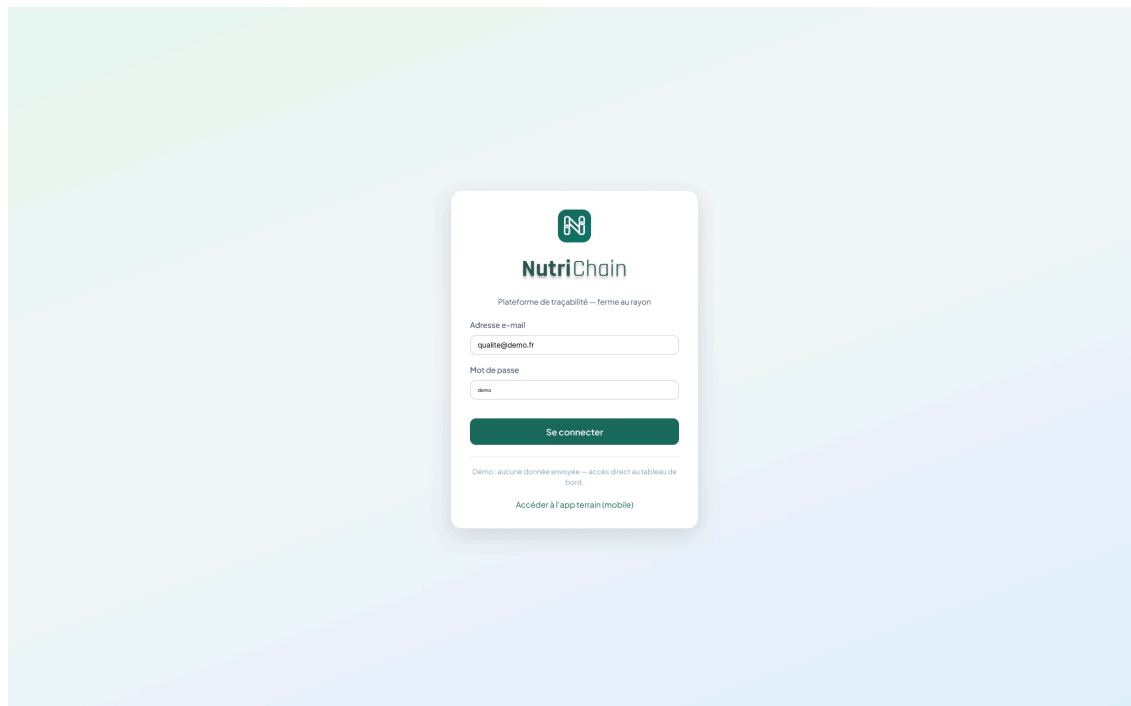
[Lien du schéma de la base de données](#)



IV. Maquettes de l'interface web et mobile

[Lien des maquettes de Nutrichain](#)

A. Interface web



NutriChain

Tracabilité & qualité

PILOTAGE

Tableau de bord

Recherche lots

Fiche lot

Tracabilité

QUALITÉ & RISQUES

Chaîne du froid

Non-conformités

Rappels produits

RÉSEAU

Portail magasins

Intégrations

SYSTÈME

Utilisateurs

Audit & logs

Tableau de bord

Recherche globale : lot, GTIN, SSCC...

3 alertes froidDéconnexion

LOTS SUIVIS (30 j)

12847

+4,2 % vs. période précédente

ALERTES CHAÎNE DU FROID

3

2 en investigation

RAPPELS EN COURS

1

Yaourt bio — lot ciblé

ANOMALIES OUVERTES

7

3 quarantaine active

SYNC: INTÉGRATIONS

99%

Dernier flux WMS il y a 2 min

Activité récente (EPCIS)

Aujourd'hui - 14:32

ObjectEvent — réception

SSCC 00376123456789012345 - Site Bretagne Nord

Aujourd'hui - 13:05

Transformation — découpe

Lot L-2025-08912 - conforme HACCP

Hier - 18:40

Aggregation — palette

EPCIS 2.0 - expédition vers RDC Île-de-France

À traiter

Validation qualité — 5 bons de réception en attente de visa.

Rappel RAP-2025-014 — retrait magasin à 78 % — voir le suivi

NutriChain v2.4 - QSI / EPCIS

NutriChain

Tracabilité & qualité

PILOTAGE

Tableau de bord

Recherche lots

Fiche lot

Tracabilité

QUALITÉ & RISQUES

Chaîne du froid

Non-conformités

Rappels produits

RÉSEAU

Portail magasins

Intégrations

SYSTÈME

Utilisateurs

Audit & logs

Recherche de lots

Recherche globale : lot, GTIN, SSCC...

3 alertes froidDéconnexion

Recherche de lots

Filtres avancés — GTIN, lot, SSCC, produit, site, dates, statut.

GTIN

N° lot

SSCC

Site

Statut

356007...

L-2025-08912

0037612...

Tous

Tous

Appliquer

Lot	Produit	GTIN	Site	Statut	Dernière temp.
L-2025-08912	Yaourt nature bio 4x125g	3560078891234	Bretagne Nord	Conforme	3,8 °C
L-2025-08801	Emmental tranché	3560078456789	RDC IDF	Surveillance	5,1 °C
L-2025-08744	Salade barquette	3560078123456	Loire	Quarantaine	9,2 °C

NutriChain v2.4 - QSI / EPCIS

NutriChain

Tracabilité & qualité

PILOTAGE

Tableau de bord

Recherche lots

Fiche lot

Tracabilité

QUALITÉ & RISQUES

Chaîne du froid

Non-conformités

Rappels produits

RÉSEAU

Portail magasins

Intégrations

SYSTÈME

Utilisateurs

Audit & logs

Tracabilité

Recherche globale : lot, GTIN, SSCC...

3 alertes froidDéconnexion

Arbre de tracabilité

Vue amont / aval — matières premières vers produits finis et expéditions.

AMONT — MATIÈRE

Lait cru — Ferme Les Aubépines

Lot MP-2025-441

↑

TRANSFORMATION

Pasteurisation + conditionnement

HACCP valide

↑

AVAL — PRODUIT FINI

L-2025-08912 — Yaourt nature bio

EPCIS Aggregation

NutriChain v2.4 - GSI / EPCIS

B. Interface mobile



NutriChain

Traçabilité agroalimentaire

Connexion sécurisée

E-MAIL PROFESSIONNEL

MOT DE PASSE

☒ Rester connecté [Mot de passe oublié ?](#)

Se connecter

SSO entreprise

Chiffrement TLS · Session conforme RGPD

Bonjour,

● En ligne

Marie L.

· Logistique — Site Lyon

SCANS AUJOURD'HUI

47

EN ATTENTE SYNC

0

ALERTES ACTIVES

2

RÉCEPTIONS

3

ACTION PRINCIPALE



Scanner un lot

GTIN, SSCC, datamatrix — lecture rapide



Réception

Marchandise entrante



Mouvement

Stockage / expédition



Alertes froid

2 incidents



Quarantaine

Lots isolés



Chaîne du froid

Palette SSCC 00 3761... — +4,2 °C



Accueil



Scan

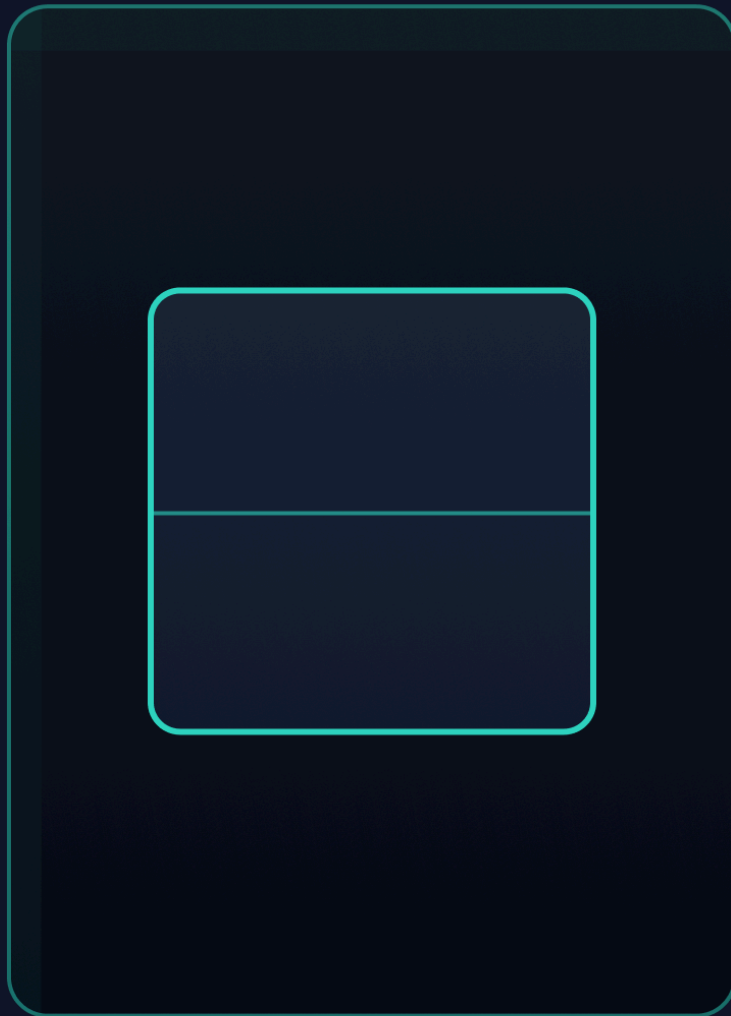


Sync



Profil

← Retour **Scanner**



Placez le code-barres ou le datamatrix dans le cadre –
lecture quasi instantanée

SAISIE MANUELLE (SSCC / LOT / GTIN)

376112345678901234

Simuler un scan réussi


Accueil


Scan


Sync


Profil

V. Mise en oeuvre de la partie infrastructure - CI/CD

A. Exemple de jeu de tests

Un premier jeu de test basique est présent pour initier un début de CI/CD sous forme de démo. Ce jeu de test se base sur des méthodes globales présentes dans un grand nombre de projet dont Nutrichain. Pour chaque méthode type utilitaire, sa version de test existe.

 checkApiKey
 createFile
 errorHandler
 formatDateError
 logFunction
 logger
 returnSuccess
 sanitizeStringData
 validateData

Name	
 ..	
 frenchError.ts	
 validateData.model.ts	
 validateData.test.ts	
 validateData.ts	

```

import { describe, it, expect } from 'vitest';
import vine from '@vinejs/vine';
import { validateData } from './validateData';

describe('validateData multiple errors', () => {
  const schema = vine.object({
    name: vine.string(),
    age: vine.number().min(18),
  });

  it('should throw both "required" and "min" errors when both fields
are invalid', async () => {
    const invalidData = { age: 16 };

    try {
      await validateData(schema, invalidData);
    } catch (error: any) {
      expect(error.status).toBe(400);
      expect(error.error).toEqual(expect.arrayContaining([
        { field: 'name', rule: 'required', message: "Ce champ
est obligatoire." },
        { field: 'age', rule: 'min', message: "La valeur doit
être supérieure ou égale à 18." }
      ]));
    }
  });

  it('should throw "required" error when "name" is missing', async ()
=> {
    const invalidData = { age: 20 };

    try {
      await validateData(schema, invalidData);
    } catch (error: any) {
      expect(error.status).toBe(400);
      expect(error.error).toEqual(expect.arrayContaining([
        { field: 'name', rule: 'required', message: "Ce champ
est obligatoire." }
      ]));
    }
  });

  it('should throw "min" error when "age" is less than 18', async ()
=> {
    const invalidData = { name: 'John', age: 16 };
  }
});

```

```

    try {
      await validateData(schema, invalidData);
    } catch (error: any) {
      expect(error.status).toBe(400);
      expect(error.error).toEqual(expect.arrayContaining([
        { field: 'age', rule: 'min', message: "La valeur doit
être supérieure ou égale à 18." }
      ]));
    }
  });

  it('should pass validation when all fields are valid', async () => {
    const validData = { name: 'John', age: 20 };

    const result = await validateData(schema, validData);

    expect(result).toEqual(validData);
  });

  it('should throw "string" error when "name" is not a string', async
() => {
    const invalidData = { name: 123, age: 20 };

    try {
      await validateData(schema, invalidData);
    } catch (error: any) {
      expect(error.status).toBe(400);
      expect(error.error).toEqual(expect.arrayContaining([
        { field: 'name', rule: 'string', message: "Ce champ doit
être une chaîne de caractères." }
      ]));
    }
  });

  it('should throw "number" error when "age" is not a number', async
() => {
    const invalidData = { name: 'John', age: 'twenty' };

    try {
      await validateData(schema, invalidData);
    } catch (error: any) {
      expect(error.status).toBe(400);
      expect(error.error).toEqual(expect.arrayContaining([
        { field: 'age', rule: 'number', message: "Ce champ doit
être un nombre." }
      ]));
    }
  });

```



```
        ]));  
    }  
  });  
});
```

B. Début de CI/CD de Nutrichain API

```
name: API CI/CD  
  
on:  
  push:  
    branches: [ main ]  
  pull_request:  
    branches: [ main ]  
  
jobs:  
  ci:  
    name: API CI  
    runs-on: ubuntu-latest  
  
    services:  
      postgres:  
        image: postgres:15  
        env:  
          POSTGRES_USER: nutrichain  
          POSTGRES_PASSWORD: nutrichain-pwd  
          POSTGRES_DB: nutrichain_db  
        ports:  
          - 5432:5432  
        options: >-  
          --health-cmd="pg_isready"  
          --health-interval=10s  
          --health-timeout=5s  
          --health-retries=5  
  
    env:  
      DATABASE_URL:  
postgres://nutrichain:nutrichain-pwd@localhost:5432/nutrichain_db  
      NODE_ENV: test  
  
    steps:  
      - name: Checkout code  
        uses: actions/checkout@v4  
  
      - name: Setup Node.js
```

```
  uses: actions/setup-node@v4
  with:
    node-version: 22
    cache: npm

- name: Install dependencies
  run: npm ci

- name: Generate Prisma client
  run: npx prisma generate

- name: Apply database migrations
  run: npx prisma migrate deploy

- name: Run unit tests
  run: npm run unit:test

- name: Build API
  run: npm run build

cd:
  name: API CD
  runs-on: ubuntu-latest
  needs: ci
  if: github.ref == 'refs/heads/main'

steps:
- name: Checkout code
  uses: actions/checkout@v4

- name: Setup Node.js
  uses: actions/setup-node@v4
  with:
    node-version: 22
    cache: npm

- name: Install dependencies
  run: npm ci

- name: Build for production
  run: npm run build

- name: Upload API artifact
  uses: actions/upload-artifact@v4
  with:
    name: api-build
```

path: **dist**

VI. Tâches et suivi du livrable 2

Tickets	January		February		March		
Sprints	Sprint 5	Sprint 6	Sprint 7	Sprint 8	Sprint 9	Sprint 10	Sprint 11
> SCRUM-44 MVP optionnel - Démarra... TERMINÉ							
> SCRUM-50 Créer les schémas C4 niveau 1 et 2 (Hug... TERMINÉ							
> SCRUM-53 Dessiner le processus de réception des produits (Hugo) TERMINÉ							
> SCRUM-51 Rédiger la justification de l'architecture (Yoann) TERMINÉ							
> SCRUM-54 Dessiner le processus de rappel produit TERMINÉ							
> SCRUM-52 Dessiner le processus de transformation d'un l... TERMINÉ							
> SCRUM-56 Imaginer la base de données TERMINÉ							
> SCRUM-57 Dessiner le schéma complet de la base de donnée... TERMINÉ							
> SCRUM-58 Commencer les maquettes de l'application web (Hugo) TERMINÉ							
> SCRUM-59 Continuer les maquettes web (Hug... TERMINÉ							
> SCRUM-60 Préparer les fichiers de test pour la CI/C... TERMINÉ							
> SCRUM-61 Organiser le dossier de tes... TERMINÉ							
> SCRUM-63 Créer les maquettes de l'application mobile (Hugo, Paul) TERMINÉ							
> SCRUM-64 Finaliser la CI/CD et faire les captures TERMINÉ							
> SCRUM-65 Livret de suivi SCRUM (Hugo) TERMINÉ							
> SCRUM-84 Vérifier que tous les documents L2 sont présents TERMINÉ							
> SCRUM-86 Assembler et rendre le Livrable 2 complet (Aide possible) TERMINÉ							
> SCRUM-88 Débuter Micro Servic...							
> SCRUM-91 Configurer Docker Compose pour lancer tous les services							
> SCRUM-93 Écrire le README avec les 3 commandes de démarrage (Hugo)							
> SCRUM-95 Développer le service de gestion des lo...							